Requirements

1. Food Price Inflation Dashboard

    1.1. Functional Requirements

        1.1.1. System must retrieve CPI (Consumer Price Index) data
        1.1.2. The system must generate line charts showing monthly prices for each food.
        1.1.3. Users must be able to select a time range.
        1.1.4. Users must be able to filter the categories of food.
        1.1.5. System must show numerical price for each item in SGD, to 2 decimal places.

    1.2. Non-Functional Requirements

        1.2.1. Response Time: Display updated chart < 5 seconds after user selects new range.
        1.2.2. Accuracy: Price values must be within ± 0.05 SGD of CPI dataset.
        1.2.3. Availability: Function must be accessible 24/7 with 99% time.

2. Import Dependency Score

    2.1. Functional Requirements

        2.1.1. System must retrieve food import data by type.
        2.1.2. The system must classify each food as Low (0-50%), Medium (51-80%), High (81-100%) import dependency.
        2.1.3. System must show food item, import dependency percentage, and risk level.

    2.2. Non-Functional Requirements

        2.2.1. Response time: Display import dependency table < 5 seconds after dataset load.
        2.2.2. Accuracy: Import percentage calculation should be within ± 0.5% of dataset.
        2.2.3. Availability: Function must be accessible 24/7 with 99% time.

3. Budget Meal Planner

    3.1. Functional Requirements

        3.1.1. User must be able to set budget in SGD.
        3.1.2. User must be able to select meal rules.
        3.1.3. User must be able to select dietary rules (halal/vegetarian).

3.1.4. User must be able to select calorie target range.
3.1.5. User must be able to select eat out or cook at home.
3.1.6. System must create a day/week meal plan with itemized costs and food options.
3.1.7. Users must be able to click “Swap” to replace a meal with a cheaper or alternative option that still fits the meal rules.

3.2. Non-functional Requirements

3.2.1. Response time: Generate meal plan in < 5 seconds.
3.2.2. Accuracy: Price estimates within ±10% of actual CPI-adjusted baseline.
3.2.3. Reliability: System must store meal plan data until user exits or downloads.
3.2.4. Transparency: All substitutions/swaps must include a clear explanation (e.g., “Tofu saves $2.10 vs chicken”).

4. Smart Hawker Swap

4.1. Functional Requirements

4.1.1. The system must retrieve average dish prices from CPI / NEA datasets.
4.1.2. The system must detect food categories with high inflation.
4.1.3. The system must recommend alternative hawker meals within a 3 km radius that are cheaper.
4.1.4. The system must display estimated savings when swapping dishes.
4.1.5. The system must allow users to filter suggestions by cuisine type.
4.1.6. The system must provide location details and directions to the suggested hawker stalls.

4.2. Non Functional Requirements

4.2.1. Usability
4.2.1.1. The app must present results in a simple, mobile friendly UI with minimal steps.
4.2.1.2. The system must support both light and dark modes.

4.2.2. Performance
4.2.2.1. Search results must be generated in less than 5 seconds.

4.2.3. Reliability
4.2.3.1. The system must handle missing data gracefully (eg: no, CPI update for a month).

4.2.4. Scalability

4.2.4.1. Must support at least 10,000 concurrent users without slowdown.

4.2.5. Security and Privacy

4.2.5.1. The system must not store precise GPS coordinates after the session ends.

4.2.6. Data Quality & Accuracy

4.2.6.1. Must refresh prices from government datasets at least once daily.

4.2.6.2. Location searches must be accurate to within 300 m.

5. Grocery Basket Optimizer

5.1. Functional Requirements

5.1.1. The system must allow the user to enter a shopping list.

5.1.2. The system must go through each grocery item and map it to a product category and a unit.

5.1.3. The system must let the user specify constraints such as budget and dietary tags.

5.1.4. The system must suggest cheaper substitutions at category level.

5.1.5. The system must calculate total basket cost and display % savings.

5.1.6. The system must be able to lock certain grocery items so they are not substituted.

5.1.7. The system must allow the user to re-run the optimizer after adjusting constraints.

5.2. Non Functional Requirements

5.2.1. Response Time

5.2.1.1. The optimizer must generate a basket for at least 30 items within 5 seconds.

5.2.2. Reliability

5.2.2.1. If store price is missing, the system must fall back to CPI averages.

5.2.2.2. The saved basket optimizer must be retrievable anytime.

5.2.3. Usability

5.2.3.1. The basket optimizer must support both light and dark modes.

5.2.4. Security & Privacy

5.2.4.1. No personal identifiers must be stored. Basket data should be linked to a user ID.

5.2.5. Data Accuracy

5.2.5.1. Unit conversions must be accurate within 1% error.

5.2.5.2. Prices must be updated at least once daily.